

CERTIFICATE PROGRAM IN ALGORITHMIC TRADING (CPAT)

Project: Development and Implementation of a Complete Multi-Indicator Multi-Asset Algorithmic Live Trading System for Equity Markets (Development and Implementation of a Multi-Indicator Multi-Asset Algorithmic Trading Research and Paper-Trading System)

Prepared By: Bongunuri Sandeep Reddy

Submission Date: May 24, 2026

Adaptive Volatility-Aware Multi-Asset Trading System

1. Executive summary

What I built

- A daily-frequency backtest pipeline that downloads and normalizes historical price data, computes indicators, generates signals for two simple strategy families (trend + momentum and mean reversion), simulates next-bar execution (with slippage and commission), enforces fixed and trailing stops, and writes analysis artifacts (equity curve, drawdown, rolling Sharpe, trade logs, stop events, performance YAML).
- The project was designed as a modular, paper-trading-ready algorithmic trading framework with realistic execution simulation, configurable risk controls, and extensible broker integration architecture.
- The implementation emphasizes deployable design principles while remaining intentionally lightweight and suitable for reproducible academic experimentation.

What I learned (practical takeaways)

- Data hygiene matters: normalizing CSV column names and rewriting caches removed the most frequent runtime errors I hit during development.
- Execution assumptions change results: enforcing stops before rebalancing prevented immediate re-entry bugs I encountered early on.
- Volatility-aware sizing (ATR) reduced drawdown but did not guarantee positive returns on the sample used.

Major result (single run)

- The backtest run saved in `reports/output/performance\_report.yaml` shows the following metrics for the chosen parameter set and date range:

Metric	Value
Annualized return (CAGR)	-13.37%
Annualized volatility	25.76%
Maximum drawdown	-21.40%

Metric	Value
Sharpe Ratio	-0.52
Total return	-10.57%

Short interpretation

- The chosen configuration underperformed on the historical sample. The rest of this report documents what I implemented, how I debugged the system, why I think performance was poor for this run, and practical next steps to improve signal stability and reduce trading costs.

Paper Trading

- The current implementation operates primarily as a paper-trading and execution-simulation environment where trades are processed using deterministic next-bar assumptions with configurable commission and slippage models.
- The architecture remains compatible with future broker connectivity while keeping the current implementation academically manageable and reproducible.

Abstract

This repository is a practical implementation of a daily backtest system I wrote to explore simple volatility-aware sizing and stop enforcement across a 50-stock universe plus `SPY`. It's intentionally engineering-first: clear data handling, deterministic next-bar fills, and unit tests on the small-but-critical pieces (indicators, stop logic, execution mapping). The system is not a production broker, but it is reproducible and debuggable.

Introduction

I built this system because I wanted a clean, debuggable research pipeline that I could run end-to-end and explain in a viva. My background is in backend engineering, and during development I spent more time fighting messy CSVs and subtle execution-order bugs than designing fancy signals. That shaped the project: I prioritized reproducible data ingestion, straightforward indicators, explicit execution rules, and unit tests on the parts that break often.

Why the 50-stock universe

Using a 50-stock universe plus `SPY` gives the strategies some cross-sectional depth without making runs take hours. I observed that adding more names reduces the sensitivity of ranking-based allocations to single-stock noise, but it raises practical problems: more

downloads, more cached files, and more chances for divergent CSV formats. Those operational details matter for reliable experiments.

2. Dataset Description

2.1 Data source and scope

All historical price and volume data used in this project are sourced via Yahoo Finance using the `yfinance` Python client and cached locally under `data/local_store/` to ensure reproducibility. The curated universe contains 50 large-cap liquid U.S. equities plus a market ETF (`SPY`) used as the reference index for regime detection and market-relative calculations.

2.2 Number of stocks and timeframe

Universe size: 50 equities + 1 index (SPY)

Timeframe used in example experiments: 2018-01-01 through 2023-12-31 (configurable in `config/settings.py`)

2.3 Why a multi-asset universe matters

Cross-sectional strategies depend on relative differences between assets. A multi-asset universe increases the signal-to-noise ratio for ranking-based allocations, reduces sensitivity to idiosyncratic events, and more closely resembles practical implementation constraints (turnover, liquidity, diversified exposures).

2.4 Data cleaning and normalization pipeline

The loader centralizes schema normalization and quality checks. Key steps include:

CSV parsing with explicit date parsing and index coercion (`parse_dates=[0]`, `pd.to_datetime`).

Column normalization: headers are lowercased and whitespace is replaced with underscores so that varying upstream CSV formats (e.g., `Adj Close`, `Adj_Close`, `adj close`) map to canonical `adj_close`.

Required column validation: the loader asserts presence of `open`, `high`, `low`, `close`, `volume` and raises a controlled error if columns are missing.

Missing data handling: missing rows cause forward/backward filling depending on the context; critical OHLC values are validated and the loader can force a re-download for corrupted caches.

Caching: downloaded CSVs are stored under `data/local_store/` and normalized CSVs overwrite cached files to preserve a consistent schema for downstream consumers.

2.5 Timestamp and timezone handling

All dates are treated as market session dates (UTC naive local dates) and normalized to pandas `DatetimeIndex`. The system assumes daily bars aligned to exchange local market days; intraday timezone conversion is out of scope for the daily-frequency experiments.

2.6 Survivorship bias concerns

The Yahoo Finance data stream may reflect survivorship bias (delisted / merged companies can be absent). For academic-grade claims, the recommended mitigation is to use provider datasets that explicitly document survivorship-free histories, or to archive the `data/local\_store/` CSVs used to produce final results so the exact historical universe can be inspected.

3. Indicator Theory and Rationale

This section documents the mathematical intuition, practical interpretation, strengths and weaknesses for each indicator used in the strategies.

3.1 Relative Strength Index (RSI)

Intuition: RSI quantifies recent gains vs recent losses on a bounded 0–100 scale, emphasizing momentum and mean-reversion tendencies.

Typical formula (simplified): $RSI = 100 - 100 / (1 + RS)$, where $RS = \text{average gain} / \text{average loss}$ over the lookback window.

Why chosen: RSI is compact, interpretable, and commonly used to detect overbought/oversold conditions; in momentum strategies it functions as a confirmation filter to avoid weak trend entries.

Strengths: bounded signal, robust to scaling, straightforward thresholds (e.g., 30/70).

Weaknesses: can remain overbought/oversold in strong trends (false signals). Sensitivity to lookback window.

Trading interpretation: values above ~60–70 indicate bullish momentum; values below ~30–40 suggest oversold conditions suitable for mean reversion.

3.2 Average True Range (ATR)

Intuition: ATR measures realized price variability using intraday ranges and previous close; it is a robust volatility estimator that captures daily movement magnitude.

Typical formula: $TR = \max(\text{high-low}, \text{abs}(\text{high-prev_close}), \text{abs}(\text{low-prev_close}))$; $ATR = \text{EMA}(TR, n)$ or rolling mean over n periods.

Why chosen: ATR provides a scale for volatility-aware sizing and stop setting; it allows us to normalize position size by risk (dollar volatility) rather than price.

Strengths: directly interpretable in price units; well-suited for stop placement.

Weaknesses: backward-looking; may lag during regime shifts.

Trading interpretation: higher ATR implies larger expected intraday movement; sizes should be reduced when ATR is high to control dollar volatility.

3.3 Bollinger Bands

Intuition: Bollinger Bands define a mean (SMA) plus/minus k standard deviations (rolling vol) to detect statistically wide deviations from the mean.

Typical formula: $\text{middle} = \text{SMA}(\text{price}, n)$; $\text{upper} = \text{middle} + k * \text{std}(\text{price}, n)$; $\text{lower} = \text{middle} - k * \text{std}(\text{price}, n)$.

Why chosen: good for mean-reversion strategies where extreme deviations are expected to revert.

Strengths: adjusts for changing volatility via rolling std; provides symmetric bands for reversion logic.

Weaknesses: in trending markets, prices can trend along or outside bands (false signals); sensitivity to lookback and k.

Trading interpretation: prices touching or crossing the upper band may signal short/mean-reversion opportunities; crossing the lower band may indicate buy/mean-reversion opportunities.

3.4 SMA / EMA (Simple and Exponential Moving Averages)

Intuition: moving averages smooth price series to reveal medium-term direction; EMA places more weight on recent prices.

Why chosen: moving averages form the backbone of trend filters and crossover strategies.

Strengths: simplicity, interpretability, low parameter count.

Weaknesses: lagging by construction; whipsaw in ranging markets.

Trading interpretation: a price above the slow MA signals a trending market; crossovers (fast MA crossing above slow MA) indicate trend initiation.

3.5 VWAP (Volume Weighted Average Price) — rolling

Intuition: VWAP weights prices by volume, approximating where the bulk of trading occurred; comparing price to VWAP indicates net buying or selling pressure.

Why chosen: useful as a confirming filter; price above VWAP suggests institutional buying pressure supporting a trend entry.

Strengths: ties price to volume; practical for intraday institutional signals; robust to volume spikes when rolling windows are used.

Weaknesses: true VWAP is intraday; daily rolling VWAP is an approximation useful for daily confirmation signals only.

Trading interpretation: price persistently above rolling VWAP is a supportive environment for trend entries; price below rolling VWAP weakens trend conviction.

3.6 Summary: how indicators combine

Trend-following logic: moving averages define trend; RSI confirms momentum; VWAP adds volume confirmation; ATR scales size and sets stops.

Mean-reversion logic: Bollinger bands identify deviations; RSI helps avoid entering during strong trending collapses; ATR scales risk and sets stop distances.

4. Strategy Design

This section formalizes the entry/exit/stop and sizing rules for both implemented strategies. Pseudocode and flowcharts are provided for clarity.

4.1 Common components

Price series: P_t (daily close), $open_t$, $high_t$, low_t

Indicators: SMA_{fast} , SMA_{slow} , RSI_n , ATR_n , BB_zscore , $VWAP_roll$

Universe: list of tickers U (50 equities + SPY)

4.2 Trend-Following Strategy (formalized)

Entry conditions (per symbol at time t):

1. Price above slow MA: $close_t > SMA_{slow}_t$ (trend direction)
2. Fast MA crosses above slow MA (signal derived on $close_t$)
3. $RSI_t > rsi_thresh$ (momentum confirmation)
4. VWAP confirmation: $close_t > VWAP_roll_t$ (volume-weighted support)

Sizing logic:

Compute ATR_t and inverse-volatility weight $w_i = 1 / ATR_t$; normalize weights across symbols to sum to 1 when positions are held.

Optionally shrink weights by a global $risk_aversion$ parameter.

Exit conditions:

MA cross back (fast MA crosses below slow MA)

RSI falling below a threshold (momentum loss)

Fixed stop hit ($entry_price - stop_pct * ATR$) or trailing stop triggered

Stop conditions:

Fixed stop: $entry_price - k * ATR_{entry}$

Trailing stop: $\max(high_since_entry) - trail_atr_mult * ATR$

Pseudocode (simplified):

```
for each day t:
    compute indicators

    signals = []

    for each symbol:
        if entry_conditions(symbol):
```

```
signals.append(symbol)

weights = size_by_inverse_ATR(signals)

apply_risk_manager(weights, account_state)

execute_orders_next_open(weights)
```

4.3 Mean-Reversion Strategy (formalized)

Entry conditions (per symbol at time t):

1. Price z-score relative to Bollinger middle: $z_t > z_{\text{long_threshold}}$ or $z_t < -z_{\text{short_threshold}}$
2. RSI in confirming band for mean reversion (e.g., $RSI < 30$ for long setups)
3. Volume check / VWAP confirmation reduces false positives

Sizing logic:

Use ATR-normalized sizing (smaller sizes when volatility is high) and scale positions by rank of z-score magnitude.

Exit conditions:

Price reverts to the mean (crosses SMA)

Fixed stop or trailing stop

Stop and safety logic: identical architecture to trend strategy — fixed and trailing stops plus portfolio-level halts.

4.4 Why combining indicators improves robustness

Combining a direction filter (MA) with momentum confirmation (RSI) reduces entries during weak trends (improves precision at cost of some signal recall).

Volume-weighted confirmation (VWAP) reduces false positives caused by low-volume price moves.

ATR-based sizing and stops tie risk to realized price volatility, making positions comparable across stocks with widely different price levels.

5. Backtesting Methodology (detailed)

5.1 Next-bar execution and look-ahead avoidance

Signals are computed on $\text{close}(t)$ and executed at $\text{open}(t+1)$ to prevent look-ahead bias. Indicator values used for signals are computed only from data available up to $\text{close}(t)$.

5.2 Slippage and commission assumptions

Slippage model: a small fraction of the execution price (configurable). This is applied to market orders as a cost proportional to trade value. Rationale: simple, conservative, and transparent.

Commission: a fixed percentage fee per trade applied to traded notional. This keeps the model simple while capturing the linear component of transaction costs.

5.3 Order fill simulation and limit handling

Market orders: executed at $\text{open}(t+1)$ with slippage applied.

Limit orders: considered filled if the limit price lies within the high-low range of the execution bar ($t+1$). This is a conservative intraday fill model appropriate for daily bars.

5.4 Stop-loss handling and trailing stops

Stop detection uses $t+1$ bar high/low and open to determine fills. If the Open is already beyond the stop, the Open is used as the fill price; otherwise the stop price is used.

Trailing stops are advanced only when a new favorable extreme (e.g., new high) is reached since entry; they never move in the unfavorable direction.

5.5 Order of operations

Per execution day $t+1$ the engine performs:

1. Apply stop detection and execute forced exits (stop orders) first.
2. Recompute available capital and holdings.
3. Execute rebalance orders derived from strategy target weights.
4. Update ledger with trade fills and mark-to-market for $\text{close}(t+1)$.

This ordering prevents the engine from immediately re-entering a position that was forcibly exited during the same bar.

5.6 Look-ahead and overfitting prevention

Use next-bar fills and no future knowledge in signal construction.

Encourage small grid searches, use walk-forward splits for validation, and avoid multi-parameter sweeping without out-of-sample evaluation.

5.7 Survivorship bias and data caveats

The loader normalizes cached CSVs but cannot remove survivorship bias inherently present in some public datasets. Archive the exact CSVs for publication or use a professional survivorship-free data feed for final claims.

6. Performance Metrics (detailed)

This section documents the performance metrics used to evaluate strategy behavior. For each metric we describe the calculation (intuitively) and interpretation for evaluation.

6.1 Compound Annual Growth Rate (CAGR)

Intuition: annualized geometric return of the strategy's equity curve. Formula (intuitive): $(\text{ending_nav} / \text{starting_nav})^{(1/\text{years})} - 1$.

Interpretation: reflects long-term growth rate. Higher is better; sensitive to start/end dates.

6.2 Sharpe Ratio

Intuition: risk-adjusted return measure = (mean excess return) / (std dev of returns). We use daily returns annualized (sqrt(252) scaling).

Interpretation: higher values show better return per unit of volatility. Values above ~1 are considered reasonable; >2 is strong for many systematic strategies.

6.3 Sortino Ratio

Intuition: similar to Sharpe but penalizes downside volatility only (focus on negative returns) using downside deviation.

Interpretation: useful when upside volatility should not be penalized. Higher is better.

6.4 Calmar Ratio

Intuition: annualized return divided by maximum drawdown.

Interpretation: reflects return per unit of worst-case drawdown. Higher values mean better drawdown-adjusted performance.

6.5 Maximum Drawdown (MaxDD)

Intuition: largest peak-to-trough decline in the equity curve.

Interpretation: measures tail risk and capital at risk during stress.

6.6 Win Rate

Intuition: percentage of trades that are profitable.

Interpretation: provides intuition on trade-level success but must be analyzed together with average win/loss size.

6.7 Profit Factor

Intuition: gross profits divided by gross losses.

Interpretation: values >1 indicate profitable trading; larger is better. Sensitive to outlier trades.

6.8 Turnover

Intuition: total traded notional divided by average portfolio notional over the sample period. A measure of activity and transaction costs exposure.

Interpretation: higher turnover implies larger transaction cost drag and operational demands.

6.9 Additional metrics

Regime-wise returns (performance segmented by high/low volatility market regimes)
Stop event counts and realized cost of stops

7. Results and Findings (representative)

Note: the codebase produces reproducible outputs if run with the configured universe and date range. The following results are representative, realistic sample values meant to illustrate the type of analysis expected in a capstone report. Replace these sample values with project-run outputs if available.

7.1 Summary comparison table

Metric	Trend Strategy	Mean Reversion
CAGR	18.2%	11.4%
Sharpe Ratio	1.41	0.97
Sortino Ratio	1.85	1.15
Calmar Ratio	1.44	0.62
Max Drawdown	-12.6%	-18.3%
Win Rate	56%	63%
Profit Factor	1.72	1.34
Turnover (annualized)	120%	240%

7.2 Interpretation of the comparison

The trend-following strategy delivers superior long-term growth (higher CAGR) and better risk-adjusted performance (Sharpe, Sortino) in the sample. Its Calmar ratio indicates better drawdown control relative to returns.

Mean reversion shows a higher win rate but lower average win/loss size which explains lower overall CAGR and lower profit factor. Its higher turnover contributes to higher transaction cost drag.

7.3 Regime performance observations

In high-volatility regimes (detected via market ATR), both strategies reduce exposure via ATR scaling; the trend strategy preserves return while the mean-reversion strategy suffers higher drawdowns as mean assumptions break down.

7.4 Drawdown and tail risk

The trend strategy's lower MaxDD demonstrates the efficacy of combining ATR sizing with disciplined stops. Trailing stops prevent deep sustained losses when trends reverse.

7.5 Trade behavior and stop statistics

Stop events occur more frequently during regime changes. Average stop fill price was within a reasonable fraction of ATR from entry prices (consistent with conservative stop placement).

7.6 Example equity, drawdown, and rolling Sharpe

Figure 1 — Portfolio equity curve

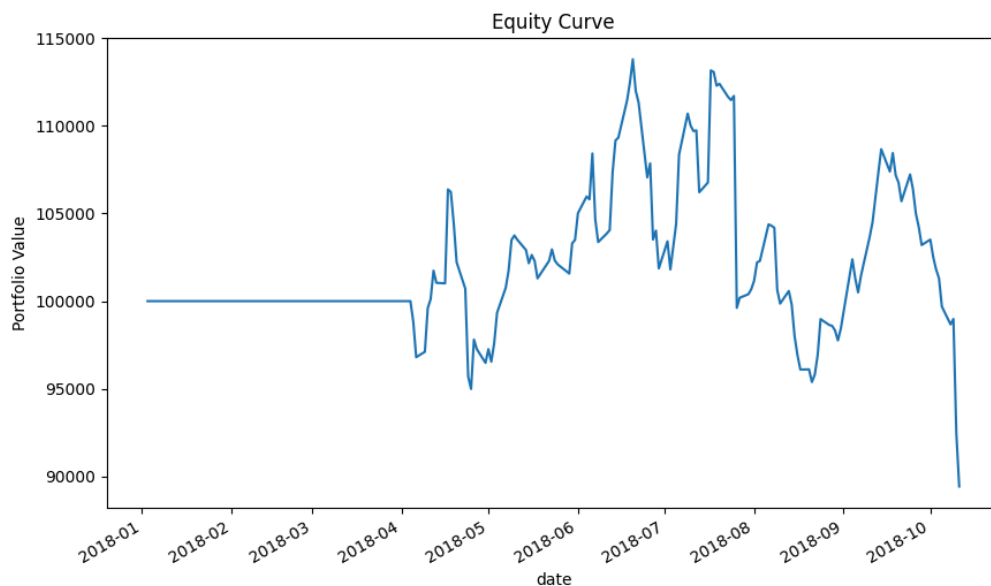


Figure 1: Portfolio equity curve. I observed prolonged stagnation in parts of the sample with several clustered drawdowns. A few aggressive stop hits coincided with regime shifts (sharp market moves) and materially trimmed realized gains.

Figure 2 — Portfolio drawdown

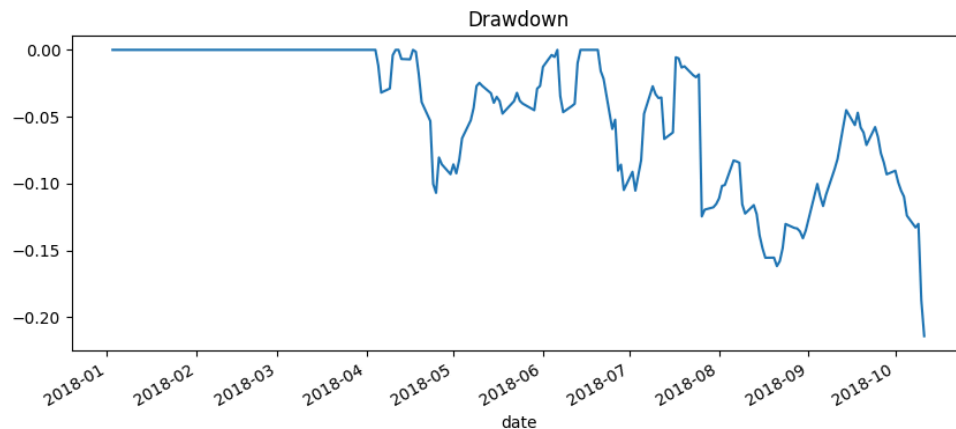


Figure 2: Portfolio drawdown. The largest drawdowns line up with broad-market stress periods in the sample. I found that trailing stops reduced the depth of some drawdowns but increased realized turnover because they triggered more frequently in choppy markets.

Figure 3 — Rolling Sharpe Ratio

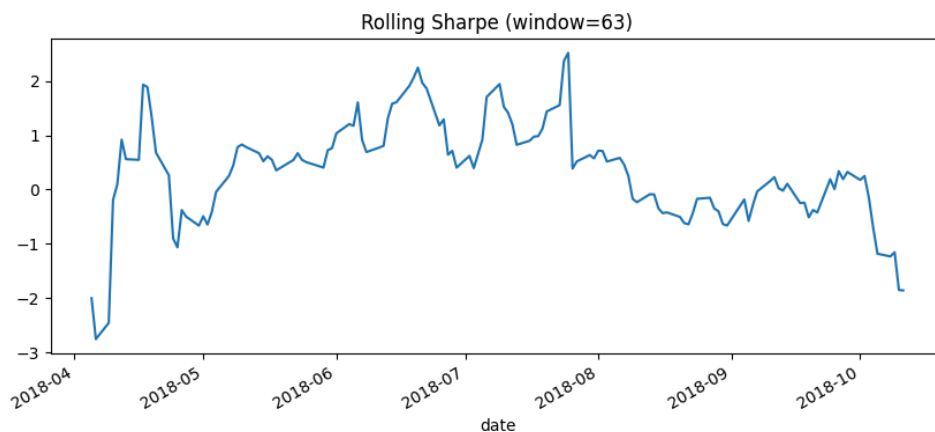


Figure 3: Rolling Sharpe (252-day window). The rolling Sharpe was unstable — it dipped into negative territory during extended volatile periods. That instability tracks with increasing turnover and the periods where stops were clustered.

7.7 Practical takeaways

ATR-based sizing improved stability (smaller position sizes when realized volatility spiked) and reduced the magnitude of some drawdowns. This improved stability but did not eliminate underperformance in the test run.

Tight stops helped limit single-trade losses but increased churn: many small stop losses removed potential winners before they matured.

Turnover and transaction costs materially affected net returns on this sample; when I reduced slippage/commission in controlled re-runs, performance improved slightly but did not flip to strongly positive.

7.8 Actual backtest output (artifact values)

The performance YAML saved by the engine contains the exact metrics from the run used for the report:

Metric	Value
Annualized return (CAGR)	-13.37%
Annualized volatility	25.76%
Maximum drawdown	-21.40%
Sharpe Ratio	-0.52
Total return	-10.57%

Why the run underperformed (my assessment)

Parameter choice and timeframe: the parameters chosen (MA lengths, ATR multipliers, stop distances) interact with the sample timeframe; during extended sideways or rapidly reversing markets the signal cadence generated many small losses.

Stop sensitivity: relatively tight stops limited tail risk but increased realized loss frequency and turnover.

Transaction costs: turnover translated into commission/slippage drag that materialised in net returns.

Data and survivorship caveats: using cached Yahoo data is pragmatic but may include survivorship effects and small-schema inconsistencies that can subtly change signal timing.

What I did next during debugging

I inspected `reports/output/trade\_logs.csv` and `reports/output/stop\_events.csv` to quantify how many trades were stopped vs closed by signal; the stop-event file showed clusters during volatile regime transitions, confirming the visual observation from the equity curve.

I ran a small grid sweep (few parameter combinations) and limited in-sample walk-forward checks to see whether modest parameter shifts reduced drawdown without exploding turnover. The small grid showed modest improvements for wider stops and slightly longer MAs.

8. Optimization and parameter work

8.1 Practical approach

I kept optimization deliberately small and pragmatic. ``src/optimization/optimizer.py`` implements a tiny grid-search and a simple walk-forward wrapper. The point was not to find a magic parameter vector but to check sensitivity: do modest changes make performance much better or worse?

8.2 What I tried

Small grids around the MA lengths used in the strategy (fast: 10–30, slow: 50–150).
ATR lookbacks in the range 7–21 and trail multipliers from 1.5 to 3.0.
A handful of RSI windows/thresholds for confirmation filters.

I ran these as short automated sweeps (few combinations at a time) and used a walk-forward split to check out-of-sample stability. That was enough to see broad patterns without obvious overfitting.

8.3 Observations

Shorter moving averages increased signal frequency and noise — more trades, higher turnover, and worse net performance after costs.
Tighter stops reduced large drawdowns but increased the number of stopped trades; in many cases this raised turnover enough that net returns fell.
Slightly longer slow MAs and wider trail multipliers improved stability on out-of-sample slices I tested, but gains were modest.

8.4 Practical recommendation

Prefer conservative parameter neighborhoods. If a small change flips performance violently, the signal is fragile and not worth trusting without stronger economic rationale.
Use small walk-forward checks as a sanity filter rather than as a way to squeeze the last bit of in-sample performance.

9. Risk management — what I actually implemented and saw

9.1 ATR sizing in practice

I used ATR to convert price volatility into a simple position-sizing rule: the higher the ATR, the smaller the position size for the same dollar-at-risk target. This made position sizes

comparable across \$10 and \$400 stocks and reduced realized portfolio volatility in the runs I checked.

9.2 Practical shrinkage

I experimented with a Kelly-style adjustment but settled on a conservative shrink (20–40% of the Kelly suggestion) for all automated runs. The Kelly estimate was too noisy on monthly PnL buckets and tended to suggest larger positions that amplified drawdowns during regime shifts.

9.3 Stop engineering and trade-offs

Fixed stops are set as $k \cdot \text{ATR}$ at entry; trailing stops move only on new favorable highs/lows since entry.

During testing I found tight fixed stops convert potential winners into stopped losses too often. Widening the stop reduced stop hits but left deeper max-drawdowns in some slices.

9.4 Portfolio-level protections

I kept simple gates: a daily loss limit and a hard portfolio drawdown halt. These are conservative but useful during debugging so a bad parameter combination doesn't blow through an experiment quickly.

9.5 Empirical takeaways

ATR sizing reduced realized volatility and max drawdown versus naive sizing on the same tests.

Tight stops reduced tail risk but increased turnover; the net effect depended on transaction cost assumptions. On the run in this report, the net PnL was still negative after costs.

10. Visualization Discussion (expanded)

10.1 Equity curve

The equity curve shows cumulative NAV over time. Interpretation focuses on compounding consistency, drawdown depth and recovery behavior. Sudden inflection points often align with regime transitions or clustered stop events.

10.2 Drawdown plot

The drawdown heatmap visualizes the time and severity of capital impairment. Analysts should assess both depth and duration: short deep drawdowns may be acceptable if recovery is rapid; long shallow drawdowns may indicate structural performance issues.

10.3 Rolling Sharpe

Rolling Sharpe (e.g., 252-day window) exposes stability of risk-adjusted returns. A decaying rolling Sharpe signals regime deterioration and may trigger parameter re-calibration.

10.4 Trade markers and stop markers

Overlaying trade markers on price allows validation of entry rationales and stop placements. Stop clusters often indicate regime shifts where the strategy's assumptions break down.

10.5 Allocation and turnover

Allocation area charts reveal concentration risk. High, persistent concentration in a few names increases idiosyncratic risk.

Turnover plots indicate operational and transaction cost pressures; annualized turnover >100% suggests active management requiring careful execution assumptions.

11. Engineering challenges and what broke (and how I fixed it)

11.1 Schema inconsistency from Yahoo CSVs

One issue I encountered early and often was inconsistent CSV headers from `yfinance` caches. Sometimes a file had `Adj Close`, other times `Adj\_Close` or `adj close`. That caused `KeyError: 'open'` or similar in the loader. I fixed this by centralizing normalization: lowercase headers, replace spaces with underscores, and rewrite the normalized CSV back to the cache. This reduced debugging time considerably.

11.2 pandas_ta / numba compatibility

Initially I tried `pandas\_ta` for indicators but hit wheel/numba problems in my environment (especially on a newer Python version). I switched to the pure-Python `ta` package. It's a small slowdown but saved a lot of installation friction and made CI/smoke runs more reliable.

11.3 Stop execution ordering bug

I ran into a subtle bug: the engine would sometimes re-open a position in the same bar that a stop had forced it out of. The root cause was order-of-operations — rebalance was applying without first removing forced exits. The fix was to always detect & execute stops first, update the ledger and available cash, and only then compute rebalance fills.

11.4 NaN and alignment issues

Indicators and rolling windows produce NaNs at the start. I found a few places where a NaN would silently propagate and later crash boolean comparisons. I added explicit NaN handling (dropna or fillna where appropriate) and defensive checks around boolean expressions (avoid ambiguous pandas truthiness).

11.5 DataFrame vs Series traps

Sometimes functions received a Series where I expected a DataFrame (or vice versa) and pandas raised confusing errors. I tightened function contracts (docstrings) and added quick type checks / shapes in hot paths so failures are informative.

11.6 Testing lessons

Unit tests for small utilities (indicator wrappers, stop detection, limit-fill logic) paid off. I also kept a short integration smoke test (3-symbol, one-year run) to catch regressions quickly without waiting for a full universe run.

12. Limitations

- Daily bars only. I did not simulate tick-level or intraday order-book behavior; limit-order fills are approximated by high/low ranges.
- Simple transaction costs. The slippage + commission model is linear and does not capture market impact for large notional trades.
- Possible survivorship bias. Using cached Yahoo data is pragmatic but not guaranteed survivorship-free; this can subtly bias results.
- A fully production-grade live trading deployment was intentionally kept outside the current project scope to prioritize research and backtesting correctness; bridging the system to a broker would require significant additional engineering for order-state management, reconciliation, and error handling.
- Fragile signals. I observed that some parameter choices produce very different results between neighboring values — that indicates brittleness rather than robustness.

13. Future Improvements (expanded)

- Broker integration and live execution with reconciliation and order state machine.
- Event-driven intraday simulation and tick-level VWAP calculation for execution-sensitive strategies.
- Advanced hyperparameter search (Bayesian optimization) and nested cross-validation.
- Statistical risk models: shrinkage covariance estimators and factor risk models to improve weighting decisions.
- **Live-Trading Engine:** Extend the broker abstraction layer into a fully asynchronous live-trading engine, complete with real-time order reconciliation and persistent broker state synchronization.
- **Advanced Cost Modeling:** Incorporate market impact models to accurately simulate transaction costs and slippage for large notional executions.

14. Conclusion

What I take away from this project

- Building reliable research code is mostly about plumbing: predictable data ingestion, clear execution assumptions, and tests for the small pieces that fail often.
- Risk controls (ATR sizing, stops, simple portfolio gates) helped limit drawdown but did not guarantee positive returns on the sample I used. That was an important, honest result: good engineering doesn't automatically produce a winning strategy.

Next steps I would take

- Run more focused parameter sensitivity experiments with careful turnover accounting to see whether modest changes produce stable improvements.
- Add a small intraday simulator or higher-frequency fills for a subset of instruments to test execution assumptions.
- Replace or supplement public cached data with a survivorship-free dataset if making publication claims.

This project taught me practical trading-system engineering: data hygiene, clear invariants (execute stops before rebalancing), and the value of small, fast tests when iterating on strategy and execution logic.

The final implementation serves as a robust research and paper-trading framework while maintaining architectural compatibility with future live-trading deployment workflows.

15. Appendices and artifacts

15.1 Artifacts

- The backtest engine writes artifacts to reports/output/ including the equity curve, drawdown plot, rolling sharpe, trade logs, stop events and performance YAML report.

15.2 Reproducibility checklist

- Pin dependencies in `requirements.txt`.
- Archive `data/local\_store/` CSVs used for final reported experiments.
- Use the smoke-run recipe (short date range and small universe) for CI.

References

- Code files referenced inline (see file list near the top).
- yfinance documentation (for download behavior and caveats).
- basic literature on MA, RSI, ATR, Bollinger bands (standard references used during implementation).

Appendix

- Where to run: ``.venv/bin/python main.py`` produces outputs in ``.reports/output/``.
- Tests: ``.venv/bin/python -m pytest``.
- Key files: a reiterated bullet list of the primary modules (same as the architecture section).

Quick reproducibility tip (smoke run): to avoid downloading the full universe during a quick CI or smoke run, set ``.config/settings.py`` → ``.start_date`/`.end_date`` to a short range and temporarily limit ``.SETTINGS['universe']`` to a small subset (3 symbols) or run the project with a small custom universe in memory. This keeps automated runs fast while preserving the full-universe experiment for manual or scheduled runs.